

Guia de Apoio — Avaliação Minishell (Questionário 42)

Este documento resume o estado atual do projeto frente ao questionário de avaliação do minishell e ajuda a responder os itens durante a defesa.

1) Resumo executivo

Com base nas verificações realizadas localmente, o projeto está **aderente na maior parte da mandatória**. Os pontos antes problemáticos foram corrigidos:

- << (heredoc) agora funciona;
- aspas simples não expandem variáveis (`echo '$USER'` imprime literal);
- após `unset PATH`, comandos sem caminho deixam de ser encontrados.

Observação importante: conforme sua orientação, a parte de **bônus deve ser desconsiderada** neste guia.

2) Checklist do questionário (resposta sugerida)

Legenda: - **SIM**: validado por teste local/evidência direta. - **NÃO**: falha confirmada. - **PENDENTE**: precisa validação ao vivo (interativo/memória).

Item do questionário	Status sugerido	Evidência prática
Compile (-Wall -Wextra -Werror)	SIM	<code>make -Bn</code> mostra <code>gcc -Wall -Wextra -Werror ...</code>
Compila sem erros	SIM	<code>make/make re</code> concluem sem erro
Makefile não relinka	SIM	<code>make && make -></code> “Nada a ser feito para ‘all’.”
Simple command & global variables	SIM	<code>/bin/ls</code> retorna status 0; global encontrada: <code>g_signal</code>
Empty command / spaces / tabs	SIM	entrada com linhas vazias/espacos não crasha
Arguments	SIM	comandos com argumentos executam normalmente
<code>echo</code>	SIM	<code>echo, echo -n OK</code>
<code>exit</code>	SIM	<code>exit -> 0, exit 42 -> 42, exit hello -> 2</code>

Item do questionário	Status sugerido	Evidência prática
Return value (\$?)	SIM (tendência)	status de comandos inválidos/validos coerente (ex.: bad ls -> 2)
Signals (Ctrl-C/Ctrl-D/Ctrl-)	PENDENTE	exige validação interativa na defesa
Double quotes	SIM	expansão em "\$USER" funciona
Single quotes	SIM	echo '\$USER' imprime literal \$USER
env	SIM	lista variáveis de ambiente
export	SIM	export X=42 refletiu em env
unset	SIM (parcial)	unset X removeu variável criada
cd	SIM	cd .., cd . com pwd coerente
pwd	SIM	retorna diretório corrente
Relative path	SIM	../bin/ls /tmp retornou status 0
Environment path (unset PATH)	SIM	após unset PATH, ls sem caminho retorna "command not found"
Redirection <, >, >>, <<	SIM	<, >, >> e << funcionando
Pipes	SIM	echo hi cat OK
Go Crazy and history	PENDENTE	histórico/setas e limpeza de buffer
Environment variables (\$VAR)	SIM (com ressalva)	exigem sessão interativa expansão em double quotes OK; regra de single quote falha

3) Evidências objetivas já coletadas

- Flags de compilação presentes:
 - gcc -Wall -Wextra -Werror -I. -c src/...
- Global variable:
 - src/core/main.c: int g_signal = 0;
- Heredoc funcionando:
 - printf 'cat << EOF\nhello\nEOF\n' | ./minishell
 - resultado: imprime hello

- Aspas simples corretas:
 - `echo '$USER' ->` imprime literal `$USER`
- `unset PATH` conforme esperado:
 - após `unset PATH`, `ls` sem caminho retorna erro de comando não encontrado.

4) Roteiro rápido para a defesa (copiar e colar)

Build / flags / relink

```
make -Bn | grep -E "gcc .* -Wall -Wextra -Werror" | head
make && make
```

Comandos básicos

```
./minishell -c "/bin/ls" >/dev/null; echo $?
./minishell -c "../bin/ls /tmp" >/dev/null; echo $?
./minishell -c "echo hello"
./minishell -c "exit 42"; echo $?
```

Aspas / variáveis

```
printf "echo '$USER'\necho \"\$USER\"\n" | ./minishell
```

PATH

```
printf "unset PATH\nls\n" | ./minishell
```

Redirecionamento e pipe

```
./minishell -c "echo hi | cat"
printf 'cat << EOF\nhello\nEOF\n' | ./minishell
```

5) O que marcar com cautela (ao vivo)

- **Signals:** testar todos os cenários do formulário manualmente.
- **History/Go Crazy:** validar setas para cima/baixo e limpeza de buffer após Ctrl-C.
- **Leaks:** o formulário exige ausência de leaks; validar com `valgrind/leaks` disponível no ambiente da avaliação.

Se você quiser, eu também posso gerar uma versão “curta” (1 página) só com respostas finais SIM/NÃO para você levar aberto durante a defesa.